CyberShield – Detect, Deceive, Decide

Agentic cyber defence with human oversight · HackAI 2026, Track 01 Agentic Cyber Defence · Version 2 · In one line: **Wazuh contains, the AI investigates, the human decides.**

Problem and target user

Security teams receive more alerts than they can investigate, many of them false positives, so real attacks get missed. Three gaps make this worse:

- **Rules don't investigate.** SIEM correlation catches only patterns written in advance and cannot explain its reasoning.
- **Blocking ends the evidence trail.** A blocked attacker returns from a new address, and the defender has learned nothing about their tools.
- **AI verdicts are hard to trust.** Many give no evidence, and attacker-written text can manipulate them.

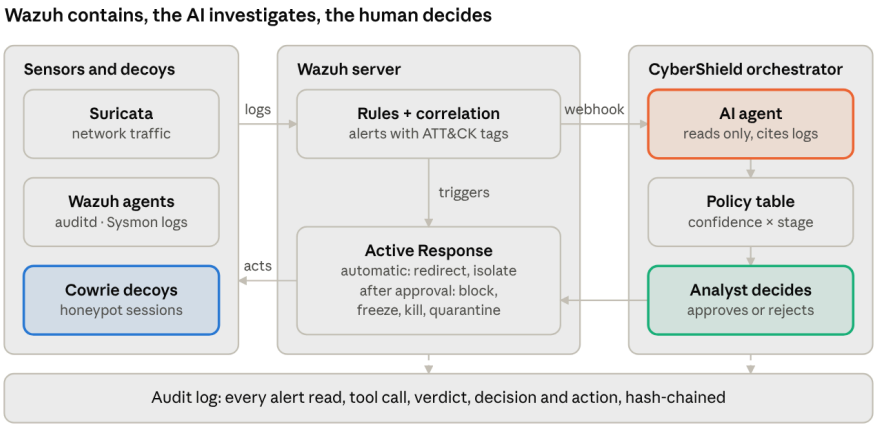
Target user: a Tier 1 analyst in a 24/7 managed security operations centre (SOC) that protects small and mid-sized organisations, who must decide quickly and defensibly.

Solution: contain first, then decide with evidence

CyberShield is one pipeline built on open-source tools (Wazuh, Suricata, Cowrie), joined by one orchestrator service into a single automated loop. Each attack type plugs in as a playbook: a sensor, a detection rule and a containment action. Containment is automatic only when it is reversible, time-limited and changes no data.

- **Stage 1, before entry: redirect into a honeypot.** After a burst of failed SSH logins, a custom Wazuh Active Response sends only that IP's new connections from port 22 to a Cowrie decoy on port 2222, and closes its connections still guessing passwords. The decoy resembles the real server, so the attacker reveals their tools and intent. Being redirected is not proof of guilt: the session is judged by behaviour (usernames tried, commands run, downloads attempted). Only contact with a standalone decoy, which nobody is ever sent to, is near-certain evidence.
- **Stage 2, after entry: contain the process.** A suspicious process moves up a ladder: observe (automatic), cut its network while it keeps running and is watched (automatic), freeze it with a memory dump (analyst), then kill it and quarantine its file (analyst). On Linux the process tree moves into a quarantine control group (cgroup) that a firewall rule cuts off; the SSH session stays up, so the attacker stays watched.

Architecture and data flow



Everything runs as one event-driven loop: one Wazuh agent per host collects every sensor, Wazuh contains at once, and a webhook pushes each alert to the orchestrator. The eight stages are the same for every attack type: **1 Sense** (Suricata `eve.json`, Wazuh agents with auditd or Sysmon, Cowrie `cowrie.json`) → **2 Detect** (Wazuh rules, including custom Cowrie rules, mapped to MITRE ATT&CK) → **3 Contain** (automatic and reversible) → **4 Investigate** (the AI agent, woken by the orchestrator) → **5 Decide** (policy table, then the analyst) → **6 Respond** (Wazuh runs the approved action) → **7 Record** (hash-chained audit log) → **8 Learn** (rejections reviewed by a person, who tunes rules and thresholds).

Agent workflow and tools

The agent (Claude, or a local model via Ollama when logs must stay on-site) works in a loop: it picks a tool, reads the result and decides what to check next. The orchestrator wakes it automatically: it groups one attacker's alerts into one incident and runs the agent once the burst settles or urgent evidence arrives, and again on new evidence, with model calls capped per incident. Its tools are `search_alerts`, `get_honeypot_session`, `check_ip_reputation` and `map_to_attack` (maps honeypot commands, which carry no ATT&CK tags, to techniques), all read-only, plus `propose_action`, which can only propose.

Its choices matter where fixed rules stop. After a download attempt in the honeypot, it pulls Suricata's record of that connection. When a username matches a real account, it checks that account's other logins. When a session looks harmless, it looks up what triggered the redirect, to support a release.

The output is one incident report: a timeline, MITRE ATT&CK techniques, **confidence** (low, medium or high) and **stage** (Probing, Trying to get in, Inside). Every claim cites a log line, and each citation is checked automatically to exist.

Decision policy and human approval

The AI never picks the action: a fixed table maps its two ratings to a recommendation, which the analyst approves or rejects on an approval screen. Only the redirect and process isolation run without an analyst; while a decision is pending they are held up to a hard cap, then escalated as overdue.

Confidence	Probing (scans only)	Trying to get in (login attempts, incl. redirected)	Inside (real foothold, process isolated)
Low	Log only	Lift the redirect (approval)	Lift the isolation (approval)
Medium	Watch list; redirect sooner	Keep in honeypot, alert analyst	Keep isolated; disable account (approval)
High	Timed block (approval)	Timed block (approval)	Freeze or kill, quarantine, disable account (approval)

Rules, not the AI, set floors under the ratings: a download attempt means at least medium, touching a standalone decoy means high, and a real login means Inside. On a critical asset, confidence counts one level higher. Allowlisted IPs and critical services are never contained automatically, and a conservative mode turns automatic containment off.

Safeguards (mapped to the OWASP Top 10 for LLM Applications 2025)

- **Prompt injection (LLM01):** log text is passed as untrusted data, output is forced into fixed fields, and the floors, the table and the analyst bound what any injected text can change.
- **Excessive agency (LLM06) and output handling (LLM05):** the agent uses a read-only Wazuh indexer account; only the orchestrator's approve path can run Active Response, through a fixed list of scripts. The approval screen escapes all log text, and analysts log in.
- **Data disclosure (LLM02):** before a cloud call, real usernames become pseudonyms and passwords are removed; default names such as root stay visible as evidence. A local model is the alternative.
- **Flooding (LLM10):** the orchestrator waits for a burst to settle before calling the AI, model calls are capped, and the system falls back to plain Wazuh alerting.
- **Tampering and learning:** a hash-chained, append-only audit log records every alert read, tool call, verdict, decision and action, and its latest hash is copied to a separate machine. Rule changes suggested by rejections apply only after a person approves them.
- **Lab safety:** all attacks stay on a host-only network with no internet; no honeypot is ever internet-facing.

Scope and evaluation

Finals demo (Phase 1, 7 Nov): the SSH brute-force playbook end to end plus Linux process isolation after a break-in, run by the orchestrator with the agent, approval screen and audit log. It runs on one laptop (32 GB RAM, RTX 5070) in VirtualBox: a Wazuh server, a victim (real SSH, Cowrie, Suricata, auditd) and a Kali attacker. Later phases add web, DoS, malware, privilege-escalation and lateral-movement playbooks, and Windows endpoints.

Baseline and metrics: we compare against Wazuh alone, counting both per incident (one attacker source in a 30-minute window). CyberShield sees only what Wazuh alerts on, so the target is the same recall with higher precision and fewer analyst decisions. We also measure evidence accuracy, time from trigger to containment, whether isolated processes can still reach the network, injection resistance, and cost and latency for Claude versus a local model.

Data: scripted lab attacks with a ground-truth log, a benign-activity script, CIC-IDS2017 Monday and Tuesday traffic replayed through Suricata (network detection only), and injection attempts typed inside Cowrie. Seven scenarios, each attack repeated at least 10 times: a full brute force; benign activity only; one attack hidden in noise; prompt injection; a non-allowlisted user's typo; a break-in with a guessed or stolen password; and a genuine user wrongly redirected.

Limitations

- A wrongly contained user fails silently until release; we report the false containment rate. One shared address (office or mobile-network NAT) can put many users in the honeypot.
- Skilled attackers may spot the decoy, for example by its different SSH host key. We never copy the real key into Cowrie, because that would expose the real server's identity.
- Network isolation does not stop damage on the machine itself, such as encrypted files; freeze is one click away. On Windows, isolation applies per program, not per process.
- A three-VM lab is smaller and cleaner than a real network, and CIC-IDS2017 has known labelling errors (Lanvin et al.), so we report lab results as lab results.

Closest prior work, Wazuh-Autopilot, runs AI agents on Wazuh with approvals; CyberShield adds deception first, honeypot behaviour as evidence, process containment and a measured Wazuh-only baseline.